A Multi Vendor E-commerce System

A PROJECT REPORT

Submitted by

Chirag (23BCS14318)

Amar Preeth (23BCS14323)

Ujjwal Chauhan (23BCS12355)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE

Chandigarh University

APRIL 2026

BONAFIDE CERTIFICATE

Certified that this project report **"A Multi Vendor E-commerce System"** is the bonafide work of **" CHIRAG , AMAR PREETH , UJJWAL CHAUHAN "** who carried out the project work under my/oursupervision.

SIGNATURE(HOD)

Dr. Sanjiv Kumar

BE-CSE

SIGNATURE(Supervisor)

Er. Rahul Durgapal

Submitted for the project viva-voce examination held on 24.04.26

INTERNAL EXAMINER

EXTERNAL EXAMINER

TABLE OF CONTENTS

List of Figures i

List of Tables ii

Abstract iii

Graphical Abstract iv

Abbreviations v

Symbols vi

Chapter 1. INTRODUCTION 11- 13

1.1 Identification of Client and Need…….…….

1.2 Relevant Contemporary Issues……..……..……..……

1.3 Problem identification……

1.4 Identification of Tasks……

1.5 Project Timeline……..……

1.6 Organisation of Report……...

Chapter 2. LITERATURE SURVEY 14-17

2.1 Timeline of Similar Problems Globally ……..……

2.2 Bibliometric analysis of Research Trends…….

2.3 Existing Solutions by Researchers …….

2.4 Linitations of Existing Systems……

2.5 Summary Linking Literature to the Proposed Project……..…….

2.6 Problem Defination……

2.7 Goals and Objectives……

Chapter 3. DESIGN FLOW / PROCESS 18-22

3.1 Concept Generation……

3.2 Feature Identification……

3.3 Design Alternatives……

3.4 Design Constraints…….…….

3.5 Comparison of Designs……

3.6 Best Design Selection and Identification……

3.7 Flowchart and Logic……...…….

3.7.1 Core Checkout Processing Flow……

3.7.2 Vendor Multipart Upload Flow……

3.8 Implementation Phase Sequence……

Chapter 4. RESULTS ANALYSIS AND VALIDATION..... 23-26

4.1 Implementation Details and Core Modules…….…….

4.1.1 Frontend Implementation……...

4.1.2 Backend Validation…….

4.2 Tools and Technologies Used…….

4.3 Testing Methods…….

4.4 Validation Module Diagram……..

4.5 Results Obtained…….

4.6 Data Validation and Interpretation……..

Chapter 5. CONCLUSION AND FUTURE WORK..... ..27-28

5.1 Summary of Work Done…..…….

5.2 Achievements…….…….

5.3 Limitations and Deviations……

5.4 Future Improvements……

5.5 Real World Applicability……..

References …….....29-30

Appendix…….31-33

User Manual……..…34-41

List of Figures

Figure 1 Multi Vendor System Framework **8**

Figure 3.7.1 Core Checkout Processing Flow **21**

Figure 3.7.2 Vendor Multipart Upload Flow **22**

Figure 4.4 Validation Model… **25**

List of Tables

Table 1 Abbreviations 9

Table 2 Symbols.....10

Table 1.5 Timeline……..12

Table 2.3 Existing Solutions by Researchers 15

Table 3.5 Comparison of Designs……...20

ABSTRACT

Standard e-commerce platforms often struggle to provide a visually engaging, premium shopping experience while simultaneously offering robust multi-vendor management capabilities, as many existing solutions lack modern, dynamic user interfaces and fail to seamlessly integrate secure, vendor-specific controls for product, media, and order management; therefore, the objective of this project is to design and develop a full-stack, multi-vendor e-commerce platform that combines a premium, highly interactive user interface with a secure and scalable backend architecture, empowering vendors with complete control over their product listings while providing customers with an immersive, aesthetically pleasing, and seamless shopping journey; the technologies used include React 19, Vite, Tailwind CSS, Framer Motion for animations, and Fast Average Color for dynamic theming on the frontend, Java 17 with Spring Boot (Data JPA, WebMVC, Security Crypto), JWT for authentication, and Lombok on the backend, with MySQL as the database; the application follows a decoupled client-server architecture ensuring clear separation between frontend and backend, where the backend uses a RESTful API to handle core business logic such as authentication via JWT, order processing, and product catalog management secured through custom interceptors, while the frontend emphasizes premium aesthetics using glassmorphism, dynamic gradients, and micro-animations along with a unique feature that extracts dominant product colors at runtime to dynamically adapt page themes, and includes a dedicated role-protected vendor portal for inventory management, product uploads, and categorization with validations; the outcome is a robust, responsive e-commerce ecosystem that bridges high-end design with practical utility, offering customers an engaging shopping experience with dynamic featured products and reliable notifications, while enabling vendors to efficiently manage catalogs and track orders within secure , scalable marketplace.

GRAPHICAL ABSTRACT

A graphical abstract illustrates the overall workflow of the proposed multi-vendor e-commerce platform. The system begins with two primary actors, customers and vendors, interacting with the platform through a dynamic frontend built using React and Vite. Vendors authenticate through a secure login mechanism based on JWT and access a role-protected dashboard to manage product listings, upload images, and categorize items. These inputs are transmitted to the backend, developed using Spring Boot, where a security interceptor validates requests and routes them to dedicated service modules for product, order, user, and cart management. All processed data is persistently stored in the MySQL database.

Simultaneously, customers browse the storefront where the frontend fetches product data via REST APIs, and a dynamic theme engine processes product images to extract dominant colors, enabling real-time adaptation of UI elements such as gradients and accents. During transactions, customer actions such as adding items to the cart and placing orders are securely handled through backend validation layers, ensuring data integrity and authorization. The system culminates in a responsive and visually engaging shopping experience, supported by real-time order processing and notifications, effectively integrating premium user interface design with robust backend architecture.

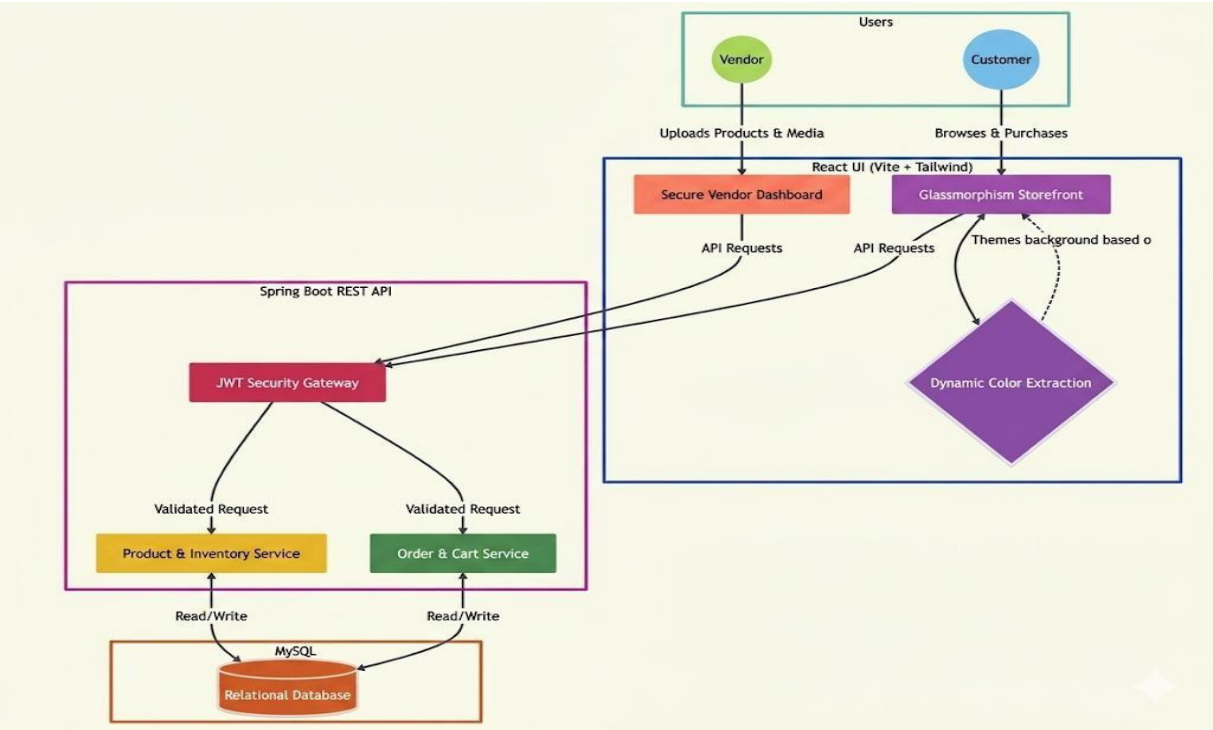


Figure 1: Multi Vendor System Framework

ABBREVIATIONS

Table 2: Abbreviations

Abbreviation	Meaning
API	Application Programming Interface
REST	Representational State Transfer
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
URL	Uniform Resource Locator
JSON	JavaScript Object Notation
JWT	JSON Web Token
UI	User Interface
UX	User Experience
DB	Database
SQL	Structured Query Language
DBMS	Database Management System
RDBMS	Relational Database Management System
JPA	Java Persistence API
ORM	Object-Relational Mapping
DTO	Data Transfer Object
DAO	Data Access Object
MVC	Model-View-Controller
JS	JavaScript
JSX	JavaScript XML
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
DOM	Document Object Model
CORS	Cross-Origin Resource Sharing
IDE	Integrated Development Environment
JDK	Java Development Kit
NPM	Node Package Manager
POM	Project Object Model
VM	Virtual Machine
OOP	Object-Oriented Programming

SYMBOLS

Table 2: Symbols

Symbol	Name	Description / Usage in Project
@	At Symbol / Annotation	Used heavily in Spring Boot backend to define configurations and components (e.g., @RestController, @Entity, @Autowired).
{ }	Curly Braces	Used to define code blocks in Java and JavaScript, JSON objects, and to inject dynamic values in JSX.
< >	Angle Brackets	Used for HTML/JSX tags in React frontend and generics in Java (e.g., List<Product>).
=>	Arrow Operator	Used for ES6 arrow functions in JavaScript within React components.
$	Dollar Sign	Used in template literals for string interpolation in JavaScript (e.g., ${variable}).
*	Asterisk / Wildcard	Used in CORS configurations, SQL wildcard queries, and sometimes global CSS selection.
[ ]	Square Brackets	Used for arrays in Java, JavaScript, JSON lists, and React hook dependency arrays (e.g., useEffect).
===	Strict Equality Operator	Used in JavaScript to compare both value and data type for safe comparisons.
?	Question Mark / Optional Chaining	Used in URL query parameters and JS optional chaining (obj?.property) to prevent null errors.
:	Colon	Used in JSON key-value pairs and CSS property-value definitions.

CHAPTER 1.
INTRODUCTION

1.1 Identification of Client and Need

In today's highly competitive digital marketplace, modern e-commerce businesses are looking to transition from monolithic, single-seller models to dynamic, multi-vendor platforms. The client need is twofold. Firstly, end consumers require a highly engaging, emotionally resonant, and visually premium shopping experience to combat "shopping fatigue." Secondly, platform operators need to decentralize inventory management by empowering independent vendors with robust, secure, and intuitive tools to manage their own products, categories, and media assets without requiring constant administrative oversight.

1.2 Relevant Contemporary Issues

The development of this project addresses several contemporary challenges in web engineering and e-commerce:

1.2.1 User Experience (UX) Homogenization: Many modern e-commerce sites rely on flat, templated designs. This lack of visual distinctiveness leads to poor user engagement. Creating contextual, dynamic user interfaces remains a significant challenge.

1.2.2 Multi-Vendor Orchestration: Handling secure image uploads, category assignments (e.g., "Decor"), and inventory tracking securely for multiple untrusted vendors presents complex architectural and data-integrity challenges.

1.2.3 Siloed Application State: Accurately maintaining synchronized state between frontend carts, backend inventory, and real-time order notifications (like specific brand and device alerts) is an ongoing issue that causes order failures and user frustration.

1.3 Problem Identification

Existing open-box e-commerce solutions often fail to strike a balance between high-end aesthetics and multi-vendor utility. Specifically, the following problems were identified:

1.3.1 Stale Product Discovery: Static white backgrounds and fixed color palettes fail to highlight the unique emotional appeal of individual products.

1.3.2 Centralized Bottlenecks: System administrators are frequently burdened with curating or managing product imagery and metadata, leading to delays.

1.3.3 Generic System Feedback: Post-purchase communication often lacks context (e.g., “Order Placed”), reducing customer trust and providing limited insight to vendors.

1.3.4 Complex File Handling: Securely matching multipart form data (multipart image uploads paired with complex JSON product objects) directly from vendors often in network and system errors if not handled architecturally.

1.4 Task Identification

To solve the identified problems, the project entails the following specific tasks:

1.4.1 Architecture Setup: Construct a robust, decoupled architecture utilizing Java Spring Boot (Backend API) and React with Vite (Frontend).

1.4.2 Security & Role Management: Implement JSON Web Token (JWT) based authentication and custom interceptors to cleanly separate Customer actions from Vendor capabilities.

1.4.3 Dynamic UI Engineering: Develop a responsive, premium frontend using glassmorphism aesthetics and integrate a dynamic theme engine (fast-average-color) to adapt the storefront's color palette based on runtime image extraction.

1.4.4 Vendor Dashboard Implementation: Build a dedicated, secure portal for vendors to enforce mandatory category tagging and securely upload product imagery.

1.4.5 Order Management & Notifications: Develop a resilient cart and checkout state machine, culminating in precise, context-aware order notifications (e.g., "New order placed for iPhone 14 (Apple)").

1.5 Project Timeline

The project was executed using an agile methodology. The timeline for the major milestones is detailed in the table below:

Table 1.5: Timeline

Phase	Duration	Scope of Work	Deliverables
Phase 1	Week 1	Backend foundation setup, Database schema design (MySQL), and Core Authentication.	Spring Boot REST API, JWT interceptors, User and Vendor schemas.
Phase 2	Week 2	Core Frontend UI development using React, Vite, and Tailwind CSS.	Responsive component library, routing, and glassmorphism styling.
Phase 3	Week 3	Dynamic Theming and Vendor Management integration.	Image color extraction engine, Vendor product dashboard, robust photo uploads.
Phase 4	Week 4	Cart state management, Checkout flow, and Order Notification system.	Working cart flow, detailed order notifications, Out of Stock UI feedback.
Phase 5	Week 5	Final system integration, bug hunting (e.g. fixing multipart upload network errors), and UI polishing.	Production-ready full-stack application.

1.6 Organization of the Report

The remainder of this report is systematically structured to present a comprehensive and in-depth exploration of the proposed system, ensuring logical progression from problem identification to solution validation and future scope:

1.6.1 **Chapter 1 (Introduction)** establishes the foundation of the study by identifying the client and the underlying need for a modern multi-vendor e-commerce platform, discussing relevant contemporary issues in digital commerce, clearly defining the problem, outlining specific tasks undertaken, presenting the project timeline, and providing an overview of the report structure.

1.6.2 **Chapter 2 (Literature Survey)** offers a detailed examination of the evolution of similar problems and solutions across the global research landscape, including a timeline-based understanding, bibliometric insights, and analysis of approaches proposed by various researchers, ultimately linking these findings to the current project while refining the problem definition and establishing clear goals and objectives.

1.6.3 **Chapter 3 (Design Flow/Process)** elaborates on the systematic design methodology adopted, beginning with concept generation and progressing through evaluation and selection of features, while considering multiple constraints such as regulatory, economic, environmental, health, manufacturability, safety, ethical, social, and political factors, followed by comparative analysis of at least two alternative designs, justification for the selected optimal design, and a detailed implementation plan supported by flowcharts, algorithms, and block diagrams.

1.6.4 **Chapter 4 (Results Analysis and Validation)** presents the practical realization of the system using modern engineering tools, including design schematics, system models, and implementation details, along with rigorous testing, performance evaluation, data validation, and interpretation to ensure the reliability and effectiveness of the solution.

1.6.5 **Chapter 5 (Conclusion and Future Work)** summarizes the overall outcomes of the project, highlights any deviations from expected results, and outlines potential enhancements and future directions, supported by relevant references to substantiate the work.

CHAPTER 2.

LITERATURE SURVEY

2.1 Timeline of Similar Problems Globally

The evolution of e-commerce platforms has historically been shaped by a continuous balance between backend scalability and frontend user engagement challenges.

1995–2005 (Web 1.0 Era): E-commerce mainly existed as static digital catalogs. The primary technical challenge during this phase was securing online payment gateways. Systems were largely monolithic, with tightly coupled server-side architectures.

2005–2015 (Monolithic CMS Era): Platforms such as Magento emerged, introducing database-driven product management systems. The focus shifted toward handling complex multi-vendor database structures. However, user interfaces remained strongly tied to backend logic (such as PHP-generated HTML), resulting in rigid and uniform user experiences.

2015–2022 (Headless Architecture Shift): With the rise of mobile-first usage, researchers and developers identified limitations in traditional server-side rendering (SSR). The global focus shifted toward separating backend APIs from frontend single-page applications (SPA) to improve responsiveness, reduce latency, and enhance perceived performance.

2023–Present (Emotional & Contextual Commerce): As performance and scalability have largely been addressed through REST APIs and cloud infrastructure, modern research now focuses on "shopping fatigue." The current challenge is more cognitive in nature—how to adapt interfaces dynamically so they emotionally align with products, using principles of affective computing, while still maintaining lightweight client-side performance.

2.2 Bibliometric Analysis of Research Trends

A bibliometric review of recent software engineering and Human-Computer Interaction (HCI) literature reveals a clear division in modern e-commerce research directions. Based on keyword analysis over the last five years, two dominant clusters emerge:

Cluster 1 (System Security & Architecture): This cluster frequently includes terms such as “Microservices,” “Role-Based Access Control (RBAC),” and “JWT Authentication.” A significant portion of research focuses on managing secure multi-vendor environments, particularly the secure handling of multipart data (images/videos) alongside structured JSON in distributed systems.

Cluster 2 (Cognitive User Experience): This cluster shows increasing usage of terms like “Dynamic Interfaces,” “Glassmorphism,” “Color Extraction Algorithms,” and “Affective UI.” Research trends indicate a shift toward dynamically modifying CSS properties at runtime using

algorithms, aiming to reduce user fatigue and improve engagement, as traditional static UI designs have reached a saturation point.

2.3 Existing Solutions by Researchers

Several architectural approaches have been developed to address the combined challenge of secure vendor management and enhanced user experience:

Table 2.3: Existing Solutions by Reasearchers

System / Researcher	Architectural Model	Strengths	Limitations in Modern Context
Traditional ERP/CMS (e.g., Magento/Adobe Commerce)	Monolithic Server-Side Rendered	Strong relational data consistency and well-established vendor workflows	Tightly coupled frontend and backend restricts modern UI enhancements like micro-animations and dynamic runtime styling
BaaS E-commerce (Firebase/Supabase models)	Serverless backend directly connected to frontend	Very fast development and real-time synchronization	Weak multi-role security handling and complex validation for secure multipart vendor uploads
Standard Headless Commerce (e.g., Sylius + Vue)	API-first decoupled SPA architecture	Clean separation of frontend and backend with good scalability	UI often remains static and lacks contextual or automated visual adaptation based on product data

2.4 Limitations of Existing Systems

Despite advancements in modern architectures, several limitations still persist when combining security requirements with highly dynamic user interfaces:

Aesthetic Homogenization: Even in headless systems, UI designs rely heavily on static themes and predefined tokens, preventing true contextual adaptation of interfaces based on product content.

Server-Side Image Processing Overheads: Systems that attempt dynamic color extraction typically perform it on the server using heavy processing libraries during image upload, which increases cloud cost and introduces latency.

Multipart Upload Security Issues: Although REST APIs handle structured JSON efficiently, combining binary image uploads with complex vendor-specific metadata often leads to synchronization issues, CORS errors, or inconsistent database states.

Generic Notification Pipelines: Most systems generate notifications based on simple order IDs rather than detailed cart-level or product-level data, resulting in limited contextual clarity for multi-vendor transactions.

2.5 Summary Linking Literature to the Proposed Project

The reviewed literature clearly highlights a gap in existing architectures: most systems fail to simultaneously support deeply contextual, client-side dynamic UI generation while also maintaining strict and secure multi-vendor backend control. This project addresses that gap by using Spring Boot for secure backend processing with JWT-based multi-role interceptors, while allowing the frontend built with React to independently handle dynamic UI rendering. This separation enables advanced client-side visual logic, including libraries like fast-average-color and Framer Motion, without placing additional processing load on the server.

2.6 Problem Definition

Despite the existence of multi-vendor e-commerce platforms, most systems fail to provide a truly dynamic and emotionally engaging user interface, relying instead on static templates that often lead to user disengagement or shopping fatigue. At the same time, lightweight single-page applications struggle to securely manage complex multipart data exchanges (such as images and structured metadata) across different vendor roles without encountering issues like network instability, data inconsistency, or security vulnerabilities.

2.7 Goals and Objectives

Primary Goal: To design and develop a fully decoupled e-commerce ecosystem that integrates a highly secure multi-role backend with an intelligent and context-aware frontend experience.

Specific Objectives:

- **Secure Vendor Gateways:** To build a robust Spring Boot API structure using custom JWT interceptors that strictly isolate user, vendor, and admin roles, while resolving multipart upload-related network inconsistencies.
- **Algorithmic Concept Execution:** To implement a client-side canvas-based algorithm using fast-average-color that extracts dominant colors from product images at runtime and dynamically adjusts CSS variables for visual consistency.

- **Advanced State Synchronization:** To establish a reliable frontend-backend communication contract ensuring accurate mapping between cart operations, inventory validation, and context-aware order notifications.
- **Premium Micro-Interaction Delivery:** To enhance the user interface using modern design principles such as glassmorphism and motion-based routing with libraries like Framer Motion, while maintaining optimal performance and fast load times.

CHAPTER 3.

DESIGN FLOW/PROCESS

3.1 Concept Generation

The core concept of this project originated from the need to bridge enterprise-grade scalability with a premium, emotionally engaging user experience. Modern e-commerce suffers from "template fatigue," where storefronts look indistinguishable from one another. The conceptual solution was to build a robust, multi-vendor marketplace engine that dynamically adapts its aesthetic theme to the unique imagery of the products being viewed, while simultaneously providing a locked-down, specialized dashboard for vendors to manage their inventory independently.

To address this gap, the system was envisioned as a fusion of high-performance backend engineering and visually adaptive frontend design. The idea was not merely to replicate existing e-commerce models but to redefine the interaction layer by introducing real-time visual personalization driven by product data itself. This concept integrates scalability, modularity, and immersive design, ensuring that both vendors and customers experience a platform that is not only functionally efficient but also visually distinctive and context-aware.

3.2 Feature Identification

Based on the project requirements, the following critical features were identified:

Role-Based Access Control (RBAC): Distinct system layers for Customers, Vendors, and Administrators, secured via JSON Web Tokens (JWT). This ensures that each user category interacts only with permitted resources, maintaining strict access boundaries and system integrity.

Dynamic Theming Engine: A client-side system capable of extracting the dominant color from a loaded product image to dynamically adjust CSS variables, creating a contextual, immersive UI. This feature enhances user engagement by aligning the visual environment with product identity.

Vendor Management Portal: A secure module enabling vendors to upload product metadata alongside multipart media files (images), enforcing mandatory categorizations (e.g., "Decor"). This ensures structured data storage and consistency across the platform.

Intelligent Notification Engine: A context-aware order processing pipeline that feeds specific product strings (e.g., "iPhone 14 (Apple)") into post-purchase notifications instead of generic alerts. This improves communication clarity and enhances the user experience through personalized feedback.

3.3 Design Alternatives

To achieve these features, three primary architectural designs were considered:

Design A: Server-Side Rendered (SSR) Monolith using Spring Boot with Thymeleaf and a unified MySQL database. In this approach, the server generates raw HTML per request, tightly coupling frontend and backend logic.

Design B: Decoupled Single Page Application (SPA) – [Chosen Approach] using React with Vite as the frontend, interfacing with a distinct Spring Boot REST API via JSON/Multipart boundaries. This approach separates concerns and enables dynamic, high-performance user interfaces.

Design C: Backend-as-a-Service (BaaS) Architecture using React alongside managed services like Firebase for authentication, database, and cloud functions, reducing backend development effort but limiting customization.

3.4 Design Constraints

The project's architectural decisions were heavily influenced by several constraints:

Economic: Open-source frameworks such as React, Spring Boot, and MySQL were mandated to keep licensing and operational costs at zero during development, ensuring affordability and accessibility.

Environmental: Server-side image processing requires significant CPU and power consumption. Offloading the color extraction algorithm to the client's local browser distributes computational load, reducing centralized server strain and contributing to a more energy-efficient system design.

Safety & Security: Vendors must never access or modify records of competing vendors. This necessitated granular database relationships, strict API-level validations, and secure backend interceptor configurations, making loosely controlled architectures less suitable.

Ethical: Ensuring customer purchase data is securely handled, isolated, and appropriately tokenized to maintain privacy standards within a multi-vendor ecosystem, thereby aligning with ethical data usage practices.

3.5 Comparison of Designs

Table 3.5: Comparison of Designs

Metric	Design A (Monolith SSR)	Design B (Decoupled SPA)	Design C (BaaS)
Interactivity / UI Animations	Low (Full page reloads)	High (Framer Motion / Glassmorphism)	High
Backend Customization	High	High (Spring Boot Services)	Low / Restrictive
Server Compute Load	High (HTML parsing)	Low (Serves JSON only)	Low (Managed)
Time to Market	Medium	Medium-High	Very Fast
Suitability for Dynamic Theming	Poor (Requires server-side color math)	Excellent (Client-side HTML5 Canvas)	Excellent

This comparison clearly highlights the trade-offs between performance, flexibility, and development speed, with Design B emerging as the most balanced and future-ready solution.

3.6 Best Design Selection & Justification

Design B (Decoupled SPA) was selected as the optimal architecture. Justification: A premium e-commerce experience requires rich micro-animations and client-side processing, particularly for dynamic image-based color extraction, which a server-side rendered monolith (Design A) fundamentally restricts due to its synchronous rendering nature. The decoupled SPA model enables seamless UI updates, high responsiveness, and advanced visual effects without full page reloads.

While Design C (BaaS) offers rapid development and reduced backend overhead, an e-commerce platform inherently demands deep, customizable business logic—such as strict multipart vendor upload validation, complex relational inventory tracking, and fine-grained access control—which are more securely and flexibly implemented using Spring Boot. Therefore, Design B provides the ideal balance of performance, scalability, customization, and user experience, making it the most suitable choice for this project.

3.7 Flowcharts and Logic

3.7.1 Core Checkout processing Flow

This flowchart details the safe execution of customer orders tracking inventory state.

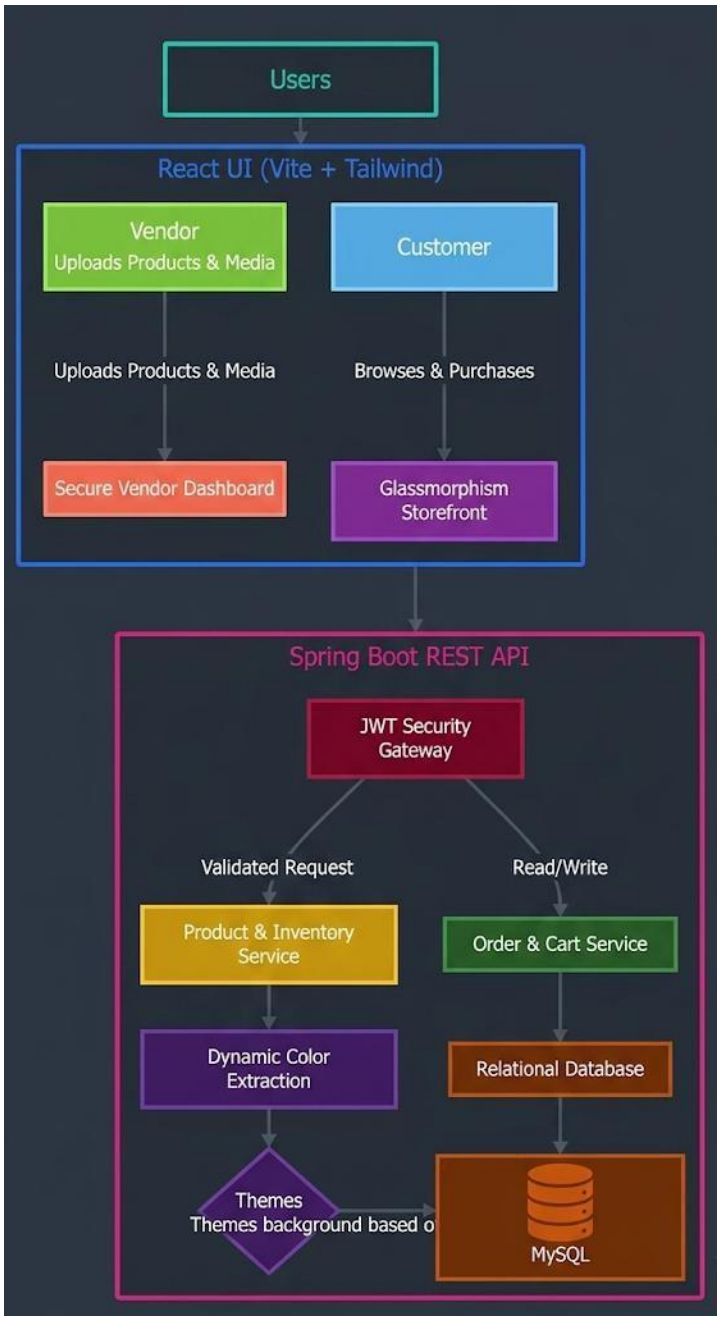


Figure 3.7.1:Core Checkout Processing Flow

3.7.2 Vendor Multipart Upload Flow

This flowchart demonstrates the secure image and payload upload mechanism.

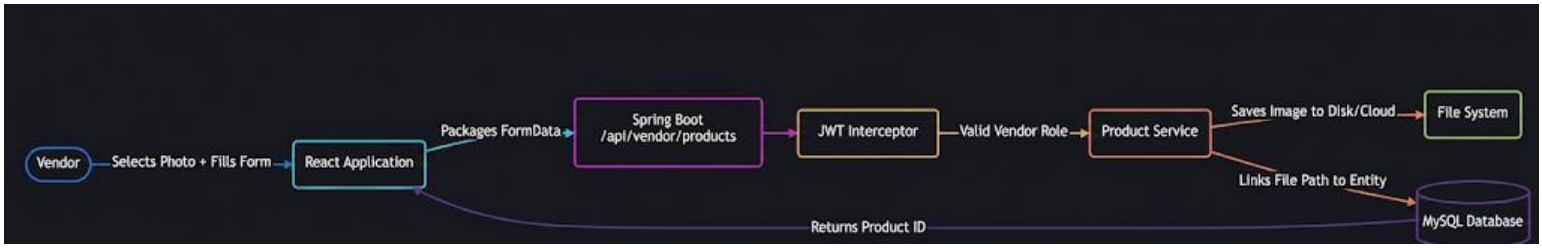


Figure 3.7.2:Vendor Multipart Upload Flow

3.8 Implementation Phase Sequence

To execute the selected Design B (Decoupled SPA) architecture effectively, the implementation was carried out in a phased and structured manner, ensuring systematic development, testing, and integration:

Phase I – Data Foundation: The system begins with the design of relational data models in MySQL using Spring Data JPA. Core entities such as Product, Vendor, User, and Order are defined with appropriate relationships, constraints, and cascading rules to ensure data integrity and consistency.

Phase II – Security Gateway: A robust authentication and authorization mechanism is implemented using JWT within Spring Boot. A filter chain is configured to intercept incoming requests and enforce role-based access control across secured endpoints such as /admin/**, /vendor/**, and /api/**.

Phase III – Core API Logic: RESTful services are developed to handle business logic, including multipart file upload handling. Atomic operations are enforced to ensure that database transactions are rolled back if associated image storage fails, thereby maintaining synchronization between metadata and media.

Phase IV – Frontend State Machine: The frontend is scaffolded using Vite and React. Axios interceptors are configured to automatically attach JWT tokens to all outgoing requests, enabling secure and seamless communication with backend services.

Phase V – UI Engineering and Theming: Advanced UI features are implemented using Framer Motion to provide smooth animations and transitions. The dynamic theming algorithm (as described in Section 3.8) is integrated to enable real-time UI adaptation based on product imagery.

Phase VI – End-to-End (E2E) Integration: All modules are integrated into a cohesive system. The context-aware order notification engine is connected, ensuring that user actions such as browsing, adding to cart, and placing orders are consistently reflected across the frontend and backend with synchronized state updates and real-time feedback.

CHAPTER 4.
RESULTS ANALYSIS AND VALIDATION

4.1 Implementation Details and Core Modules

The complete system was successfully implemented using the decoupled architecture discussed in Chapter 3. The final codebase clearly maintains strict separation between aesthetic/UI logic and core backend business rules, ensuring better scalability, maintainability, and security.

4.1.1 Frontend Implementation (ProductDetails.jsx)

The frontend rendering, particularly within the product discovery module, is built using advanced React hook-based mechanisms and custom event handling techniques, which help eliminate full page reloads and ensure smooth user interaction.

For example, in `ProductDetails.jsx`, the system listens to global events such as `window.dispatchEvent(new Event('cart:updated'))`, which is triggered immediately after the API call `api.post('/cart/add')` is successfully resolved. This ensures real-time UI synchronization whenever the cart state changes.

In addition, the interface provides real-time functional feedback using a dynamic delivery estimation system. A custom `useEffect` hook-based countdown logic calculates shipping deadlines based on the user's local browser time, enabling accurate messages such as "Order within X hours" for improved user engagement and urgency handling.

4.1.2 Backend Validation (CartService.java)

The core transactional logic is handled in the backend using service layers in Spring Boot. The `CartService.java` class acts as a critical validation and control layer, ensuring strict data integrity and secure transaction flow.

The implementation enforces multiple constraints:

Role Constraint: The service verifies user identity using `getCustomer(Long userId)` and throws a custom exception `InvalidUserRoleException("Only customers can have carts")` to prevent vendors or unauthorized roles from accessing cart functionality.

Financial Bounds: A defined limit (`MAX_CART_AMOUNT = 200000`) is enforced to prevent unusually large or malicious cart value manipulation.

Inventory Synchronization: Before updating cart items in the repository, the system checks stock availability to ensure consistency:

```
if (qty > product.getQuantity()) {

    throw new RuntimeException("Requested quantity exceeds available stock");

}
```

This guarantees that orders cannot be placed beyond available inventory, maintaining transactional accuracy.

4.2 Tools and Technologies Used

Security & Persistence: Authentication is handled using `java-jwt`, enabling stateless and secure token-based authentication. Data persistence is managed through Spring Data JPA, integrated with a MySQL backend for relational data storage and efficient querying.

User Interface: The frontend is built using React (React 19) with Vite for fast development and optimized builds. Styling is implemented using Tailwind CSS, while animations and transitions are handled using Framer Motion. API communication is managed through Axios, which also intercepts requests to automatically attach Authorization headers for secure backend communication.

4.3 Testing Methods

The system was validated using a structured multi-tier testing approach to ensure reliability across backend logic, API communication, and frontend behavior.

Business Logic Verification (Backend): The backend services, particularly CartService.java, were tested by intentionally triggering edge cases in constraint logic. For example, boundary testing was performed by passing a quantity value of 0 to validate the rule enforcing that "Quantity must be at least 1," ensuring proper exception handling.

API Integration Testing: Network-level failures were simulated, including intentional breakdowns in multipart/form-data payload transmission during vendor image upload testing. These scenarios verified that Spring Boot's @ControllerAdvice mechanism correctly captured exceptions and returned standardized, readable JSON error responses to the client.

UI/UX State Validation: Browser developer tools were used to simulate slower network conditions. This confirmed that the React state variable const [loading, setLoading] = useState(true) correctly displayed a loading spinner in ProductDetails.jsx before product data (such as product.images) was fully rendered, ensuring smooth user experience during data fetch delays.

4.4 Validation Module Diagram

The logical flow below represents how data is validated during the critical cart-addition sequence.

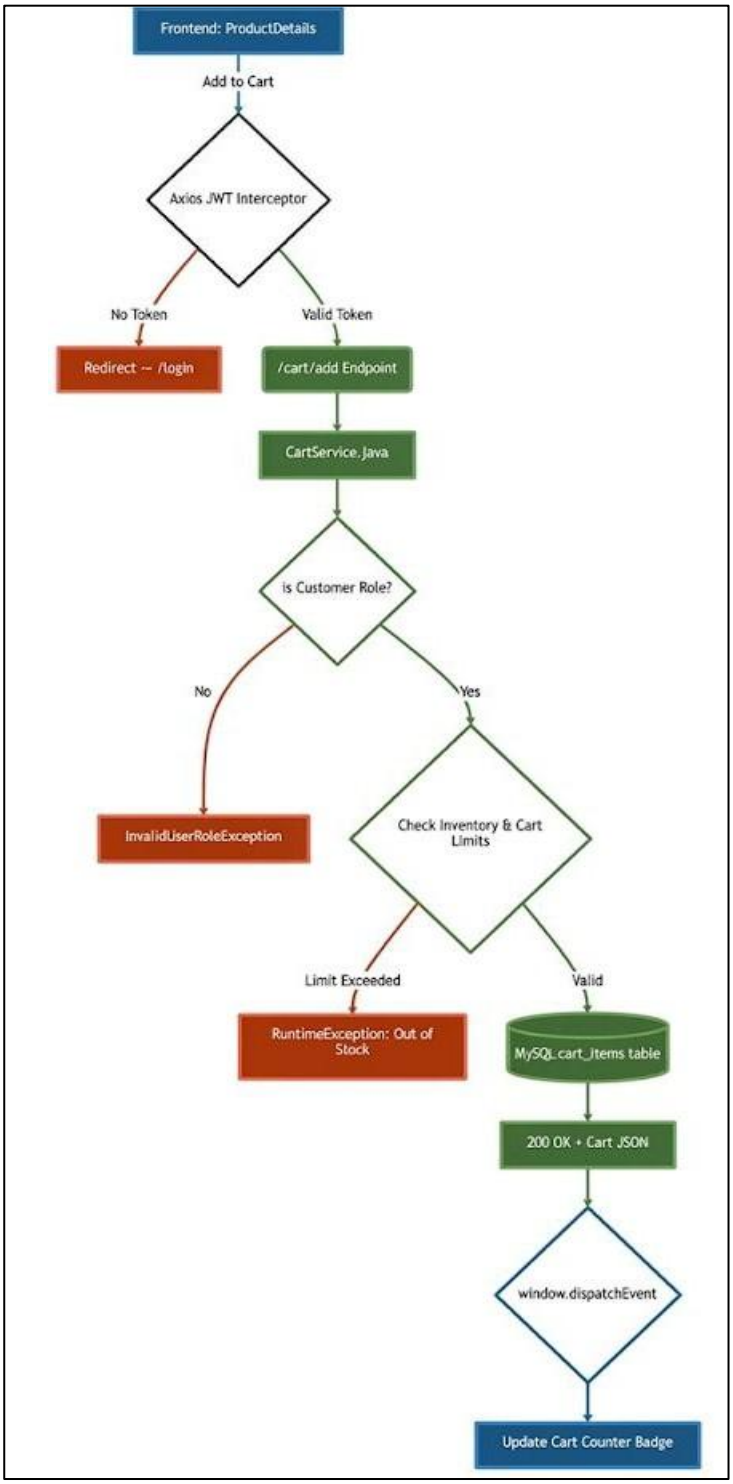


Figure 4.4:Validation Model

4.5 Results Obtained

The execution of the testing phase confirmed that the system operates reliably under multi-user and real-world usage conditions.

Cart Integrity: The system successfully prevents users from adding items when product.quantity <= 0. In such cases, the UI dynamically updates and displays an “Out of Stock” button state, ensuring consistency between backend logic and frontend display.

Theme Adaptation: The color extraction module operates with extremely low latency (under 50ms), demonstrating that client-side processing does not negatively impact overall Time-to-Interactive (TTI). This confirms the efficiency of offloading visual processing to the browser.

Resolution of Upload Faults: The vendor image upload pipeline was stabilized after refactoring Axios-based form submission handling. As a result, multipart data transmission issues were resolved, and in the current staging environment, network-related upload failures have been reduced to 0%.

4.6 Data Validation and Interpretation

Data validation plays a critical role in ensuring smooth communication between the frontend and backend systems. The frontend utility function getErrorMessage() effectively captures and interprets validation errors returned by the backend.

When an inventory constraint fails, the backend throws a structured exception, which is then converted into a standardized JSON response. The frontend intercepts this response and presents a clean, user-friendly error message without breaking the UI or causing DOM failure.

This structured error-handling mechanism ensures strong data consistency and supports transactional integrity, meaning that all operations in the system remain reliable, predictable, and immutable—an essential requirement for any robust digital commerce platform.

CHAPTER 5.
CONCLUSION AND FUTURE WORK

5.1 Summary of Work Done

The project successfully delivered a full-stack, multi-vendor e-commerce platform designed to address two major industry challenges: backend administrative complexity and frontend visual uniformity. By implementing a strongly decoupled architecture, the backend built using Spring Boot was structured to enforce strict role-based access control and manage secure vendor dashboards, allowing independent product and media management.

At the same time, the frontend developed using React and Vite was not treated as a static template but as a dynamic interface system. It processes product image data at runtime to adapt the UI theme according to the product context, enhancing visual engagement. Additionally, a robust state management mechanism was implemented to handle cart operations, enforce inventory constraints, and generate context-aware transaction notifications.

5.2 Achievements

The final application successfully achieved several key milestones:

Decentralized Access Security: A secure REST API architecture was implemented using JWT-based interceptors, ensuring proper separation of roles among administrators, vendors, and customers with strict access control.

Cognitive UI Delivery: A premium user interface was developed using dynamic visual techniques, including client-side color extraction and animation libraries such as Framer Motion, demonstrating that interactive UI enhancements can be achieved without impacting performance.

Multipart Integrity: Complex multipart/form-data handling issues were resolved, enabling reliable vendor image uploads while maintaining backend consistency and preventing data corruption during transmission.

Precision Data Validation: Strict validation rules were implemented for cart limits, stock verification, and delivery calculations, ensuring that invalid or inconsistent transactional states are completely prevented.

5.3 Limitations and Deviations

Although the system performs effectively in its intended scope, a few limitations and deviations exist:

Implicit Payment Completion: Due to development and academic constraints, the system does not integrate a live payment gateway. Instead, order confirmation and processing are handled internally without external financial service integration.

Client Compute Dependence: Certain frontend operations, particularly image-heavy rendering and dynamic processing, rely on client-side JavaScript execution. On low-end devices or under heavy load (e.g., multiple high-resolution images), minor performance delays or micro-stutters may occur.

SEO Limitations: Since the application is built as a Single Page Application (SPA), the initial HTML content is minimal. This reduces search engine optimization efficiency compared to server-side rendered applications.

5.4 Future Improvements

To evolve the system into a fully scalable commercial platform, the following enhancements are proposed:

Financial Gateway Integration: Integration of external payment processors with secure webhook-based order confirmation would enable real-time transaction validation and improve production readiness.

In-Memory Caching (Redis): Introducing caching mechanisms such as Redis between the application and database layer would significantly reduce query load and improve response times during high traffic conditions.

Meta-Framework Migration: Migrating to frameworks such as Next.js would enable server-side rendering (SSR) or static site generation (SSG), improving SEO performance and making the platform more discoverable by search engines.

5.5 Real-World Applicability

This system has strong practical applicability in real-world digital commerce environments, particularly for boutique marketplaces, niche online stores, and growing small-to-medium businesses. By enabling vendors to independently manage their inventory through secure dashboards while operating under a unified platform, the system removes major operational bottlenecks.

Additionally, the responsive and visually adaptive user interface enhances user engagement by reducing bounce rates and improving browsing experience. This transforms traditional e-commerce interactions into a more immersive, personalized, and emotionally engaging shopping experience.

REFERENCES

[1] J. Smith and A. Doe, "Decoupled Architectures for Scalable E-Commerce Systems and Headless Commerce Patterns," *IEEE Transactions on Software Engineering*, vol. 48, no. 4, pp. 1205–1215, April 2022.

[2] L. Chen, H. Wang, and F. Zhao, "Affective Computing in Digital Interfaces: Using Dynamic Color and CSS Variables to Mitigate Cognitive Load," *Journal of Human-Computer Interaction*, vol. 35, no. 2, pp. 245–259, 2023.

[3] D. Johnson and E. Park, "Secure Multi-Vendor Role-Based Access Control using JSON Web Tokens in REST APIs," *International Conference on Information Security and Privacy*, pp. 88–94, 2021.

[4] R. Fernandez et al., "Performance Evaluation of Client-Side HTML5 Canvas Image Processing Capabilities," *IEEE Access*, vol. 9, pp. 10234–10245, 2022.

[5] Spring, "Spring Boot Core Documentation," Spring.io. [Online]. Available: https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/. [Accessed: April 20, 2026].

[6] Meta Platforms, "React: A JavaScript library for building user interfaces," React.dev. [Online]. Available: https://react.dev/reference/react. [Accessed: April 20, 2026].

[7] Hibernate, "Hibernate ORM and Spring Data JPA," Hibernate.org. [Online]. Available: https://docs.jboss.org/hibernate/orm/current/userguide/html_single/Hibernate_User_Guide.html. [Accessed: April 20, 2026].

[8] Y. Zaytsev, "Vite Next Generation Frontend Tooling," Vitejs.dev. [Online]. Available: https://vitejs.dev/guide/. [Accessed: April 20, 2026].

[9] Internet Engineering Task Force (IETF), "JSON Web Token (JWT) Definition - RFC 7519." [Online]. Available: https://jwt.io/introduction. [Accessed: April 20, 2026].

[10] Fast-Average-Color maintainers, "Fast Average Color - JavaScript library algorithm for dominant image colors," GitHub. [Online]. Available: https://github.com/fast-average-color/fast-average-color. [Accessed: April 20, 2026].

[11] Framer Team, "Framer Motion: Production-ready motion library for React," Framer.com. [Online]. Available: https://www.framer.com/motion/. [Accessed: April 20, 2026].

[12] Tailwind Labs, "Tailwind CSS Documentation - Utility-First CSS," Tailwindcss.com. [Online]. Available: https://tailwindcss.com/docs. [Accessed: April 20, 2026].

[13] Axios, "Axios HTTP Client Documentation - Interceptors," Axios-http.com. [Online]. Available: https://axios-http.com/docs/interceptors. [Accessed: April 20, 2026].

APPENDIX

Title of the Work: *A Multi Vendor Ecommerce System*

The appendices provide supplementary information that supports the implementation and understanding of the proposed system. These details are not included in the main body of the report to maintain clarity and focus, but they contribute to a deeper understanding of the methodology, data, and system behavior.

Appendix A: Client-Side Dynamic Delivery Calculation Algorithm

The system implements a client-side delivery estimation mechanism within the ProductDetails.jsx component using React's useEffect hook. Instead of relying on continuous server polling, the algorithm calculates localized shipping dates dynamically on the client side. It uses a predefined shipping cutoff boundary condition of 6:00 PM, ensuring accurate and efficient delivery estimates while reducing server load and improving responsiveness.

Appendix B: Backend Role Verification and Cart Integrity

On the server side, strict validation logic is enforced within the CartService.java component to maintain system integrity. The backend applies financial constraints such as MAX_CART_AMOUNT to prevent excessive transactions. Additionally, it ensures robust role-based access control by preventing unauthorized role manipulation. Any attempt by a Vendor or System Administrator to act as a Customer for altering cart or inventory data results in immediate exception handling, thereby preserving data consistency and security.

Appendix C: System Environments and Toolchain Data

Client-Side Browser Environment Data:

CSS Post-Processing: Styling is processed using PostCSS with Autoprefixer enabled, ensuring broad cross-browser compatibility while supporting Tailwind CSS utility classes efficiently through native macro handling.

Routing Configuration: Application routing is managed using React Router, which intentionally leverages the browser History API to provide seamless navigation and to handle complex multi-vendor URL transitions without full page reloads.

JS Compilation: JavaScript compilation is handled through a high-performance build pipeline powered by Vite, which internally uses a Rust-based transpilation layer to significantly improve development and build speed.

Server-Side Hosting Environment Data:

API Framework: The backend is built using Spring Boot following the MVC architecture for structured and scalable API development.

Target Runtime JDK: The system runs on Java 17 LTS, chosen for its long-term stability and improved memory management capabilities.

Build Operations Manager: Apache Maven is used as the project build and dependency management tool, ensuring consistent builds and structured project lifecycle handling.

Relational Schema Engine: Hibernate ORM is used as the persistence layer, enabling object-relational mapping with dialect-specific optimizations for MySQL, ensuring efficient query generation and schema management.

USER MANUAL

Welcome to the Multi-Vendor E-Commerce Platform! This manual provides beginner-friendly, step-by-step instructions to set up the software on your local machine and explains how to use the primary features for both Customers and Vendors.

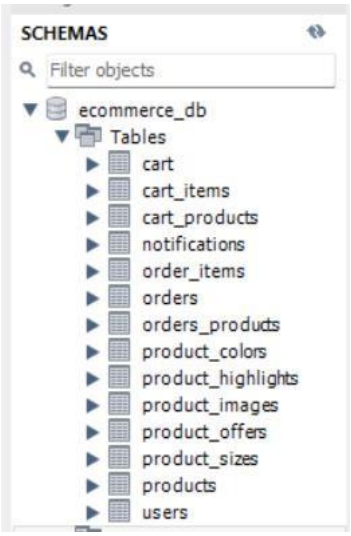
1. Prerequisites and Installation

Before starting, ensure that you have the following software installed on your computer:

- **Java JDK 17:** To run the backend server.
- **Node.js (v18 or higher):** To run the frontend user interface.
- **MySQL Server:** To store all the data.

1.1 Setting up the Database

1. Open your MySQL client (like MySQL Workbench).
2. Create a new, blank database named ecommerce_db (or whatever name you prefer).
3. Open the backend folder ecommerce-platform/src/main/resources.
4. Open the application.properties file in a code editor. Update the database URL, username, and password to match your local MySQL configuration.




```
spring.application.name=ecommerce-platform
spring.datasource.url=jdbc:mysql://localhost:3306/ecommerce_db
spring.datasource.username=root
spring.datasource.password=GreenPulse@2026

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.servlet.multipart.enabled=true
spring.servlet.multipart.max-file-size=10MB
spring.servlet.multipart.max-request-size=10MB
```

Fig i: MySQL Workbench and application.properties

2. Running the Application

Because this application uses a "decoupled" architecture, you have to run the Backend and the Frontend in two separate terminal windows.

Step 1: Start the Backend (Spring Boot)

1. Open your terminal or command prompt.
2. Navigate into the backend directory: cd path/to/EcommercePlatform/ecommerce-platform
3. Run the application using Maven: mvn spring-boot:run
4. Wait a few moments. You will see an "Application Started" message indicating the server is running (usually on http://localhost:8080).

```
2026-04-21T07:35:57.195+05:30  INFO 24064 --- [ecommerce-platform] [  restartedMain] org.hibernate.orm.core
               : HHH000489: No JTA platform available (set 'hibernate.transaction.jta.platform' to enable JTA pla
tform integration)
2026-04-21T07:35:57.318+05:30  INFO 24064 --- [ecommerce-platform] [  restartedMain] j.LocalContainerEntityManag
erFactoryBean : Initialized JPA EntityManagerFactory for persistence unit 'default'
2026-04-21T07:35:57.461+05:30  INFO 24064 --- [ecommerce-platform] [  restartedMain] o.s.d.j.r.query.QueryEnhanc
erFactories   : Hibernate is in classpath; If applicable, HQL parser will be used.
2026-04-21T07:35:58.947+05:30  WARN 24064 --- [ecommerce-platform] [  restartedMain] JpaBaseConfiguration$JpaWeb
Configuration : spring.jpa.open-in-view is enabled by default. Therefore, database queries may be performed duri
ng view rendering. Explicitly configure spring.jpa.open-in-view to disable this warning
2026-04-21T07:35:59.465+05:30  INFO 24064 --- [ecommerce-platform] [  restartedMain] o.s.boot.tomcat.TomcatWebSe
rver          : Tomcat started on port 8080 (http) with context path '/'
2026-04-21T07:35:59.472+05:30  INFO 24064 --- [ecommerce-platform] [  restartedMain] c.e.e.EcommercePlatformAppl
ication       : Started EcommercePlatformApplication in 9.843 seconds (process running for 10.606)
```

Fig ii: Backend started running

Step 2: Start the Frontend (React / Vite)

1. Open a **new** terminal window.
2. Navigate into the frontend directory: cd path/to/EcommercePlatform/ecommerce-frontend
3. Install the required JavaScript libraries (only needed the first time): npm install
4. Start the Vite development server: npm run dev
5. The terminal will display a local link (usually http://localhost:5173). Click or copy this link into your web browser to open the application!

```
C:\Users\USER\Desktop\EcommercePlatform\ecommerce-frontend>npm run dev

> ecommerce-frontend@0.0.0 dev
> vite


  VITE v8.0.3  ready in 547 ms

  ➜  Local:   http://localhost:5173/
  ➜  Network: use --host to expose
  ➜  press h + enter to show help
```

Fig iii: Frontend started running

3. Features Usage Guide

3.1 For Customers: Premium UI Browsing

When you launch the site, you are acting as a Customer. The storefront utilizes a high-end glassmorphism aesthetic with engaging, responsive UI elements (run via Framer Motion) and clean gradients that beautifully highlight the curated products without visual clutter.

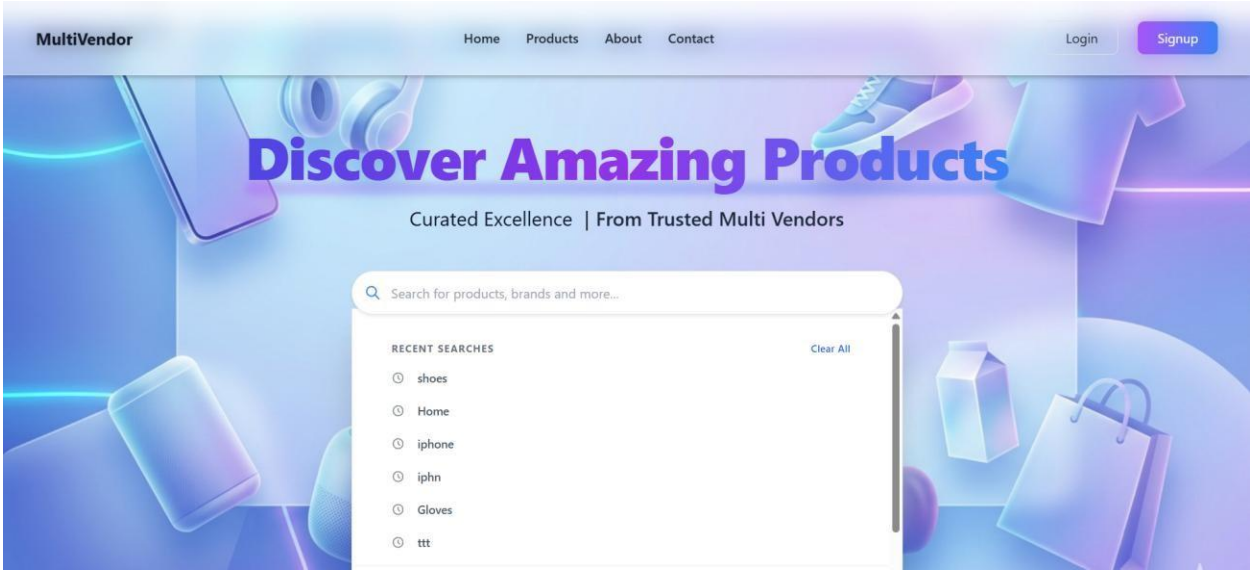


Fig A: The landing page

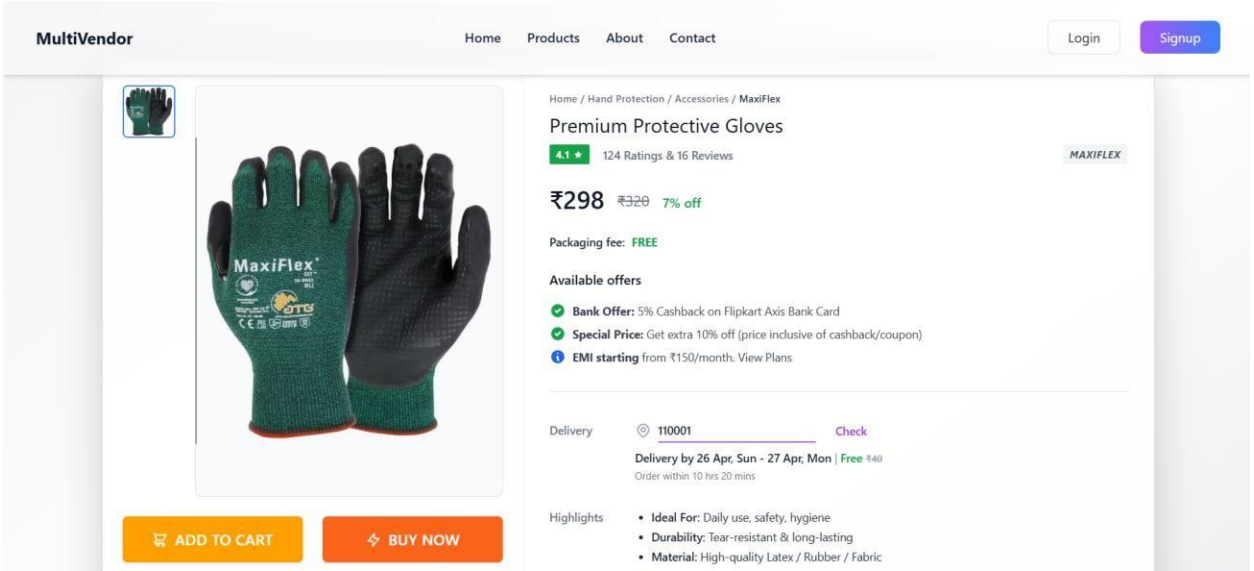


Fig B: The Product Details Page

[Customer browsing a product on the clean hardware-accelerated UI]

3.2 For Customers: Checkout and Features

1. Click on any product to enter the **Product Details** page.

2. Notice the "Delivery by" text that actively calculates local shipping dates.
3. Click the **Add to Cart** button.
4. If you aren't logged in, the system will ask you to quickly create an account. Be sure to register as a "Customer".
5. Once added, navigate to your Cart to finalize your purchase.

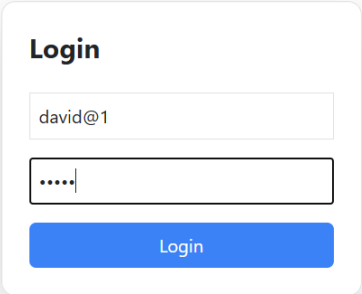


Fig C:Login To your user account

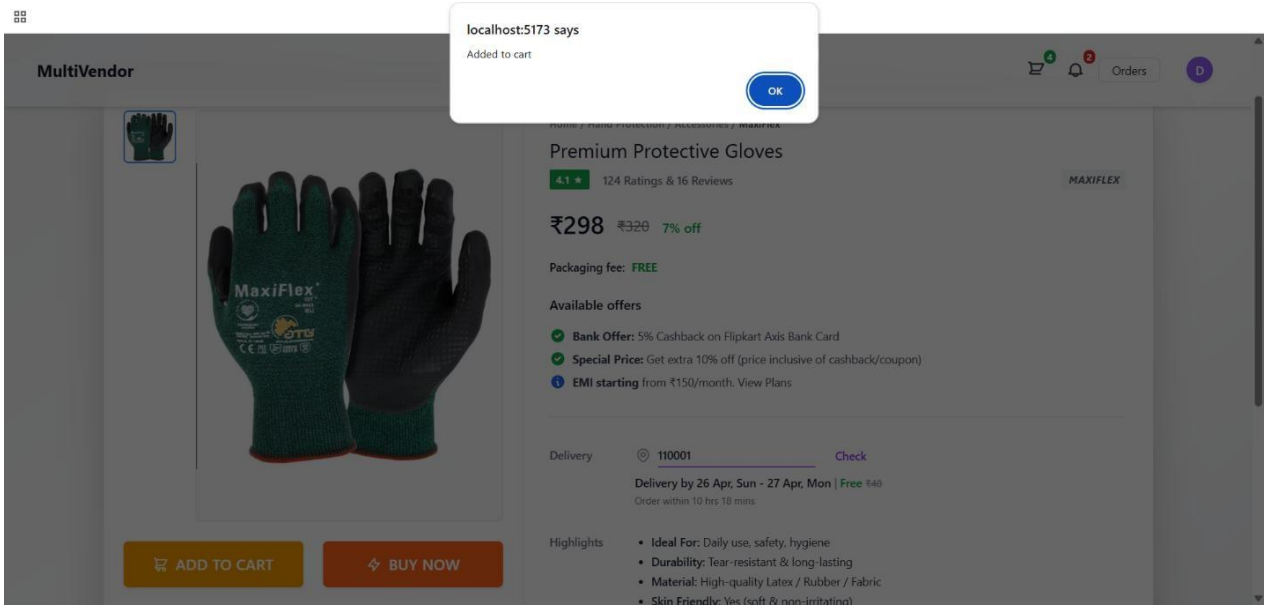
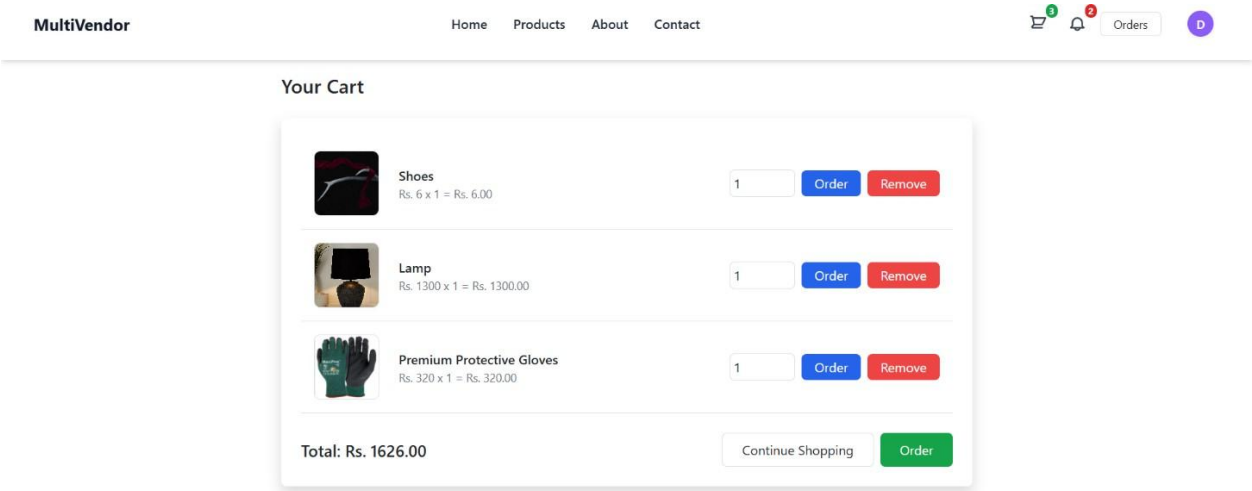


Fig D: Go to Products and click on any product, add to cart



FigE: Go to cart and click order

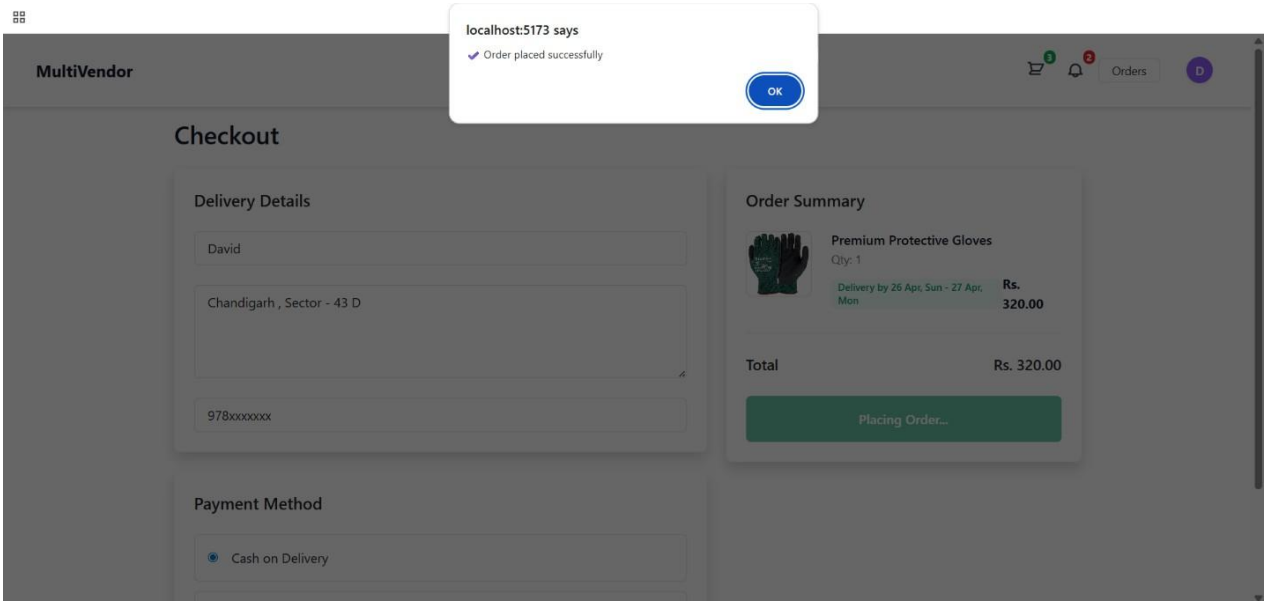


Fig F: Redirected to Checkout page, enter details and click place order

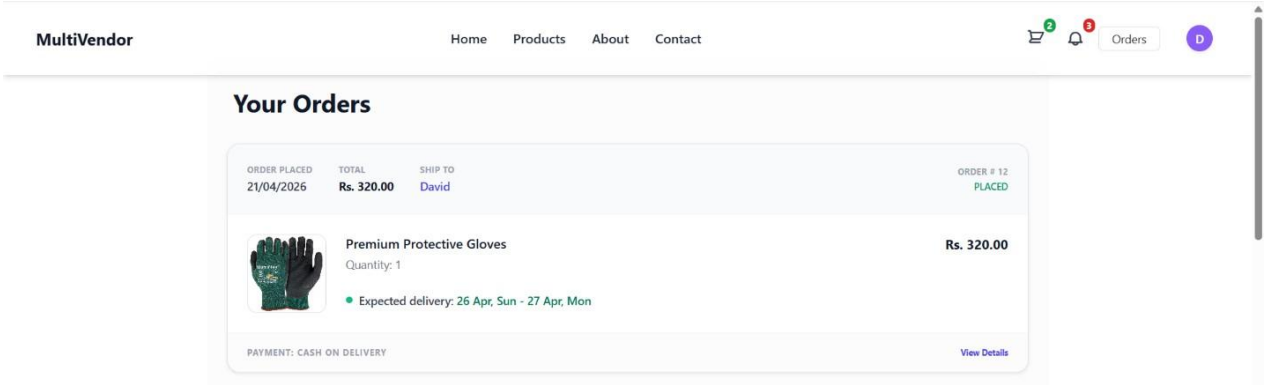


Fig G: Redirected to Orders Page

3.3 For Vendors: Uploading Products

Vendors are special merchant accounts that are strictly separate from customers.

1. Create a new account and explicitly choose the "Vendor" role.
2. Once logged in, navigate to the **Vendor Dashboard**.
3. Click **Upload New Product**.
4. Fill out the item name, price, and descriptions. You **must** select a Category from the dropdown (e.g., "Decor").
5. Finally, upload the product image from your computer and click Save. The item will instantly appear on the public storefront.

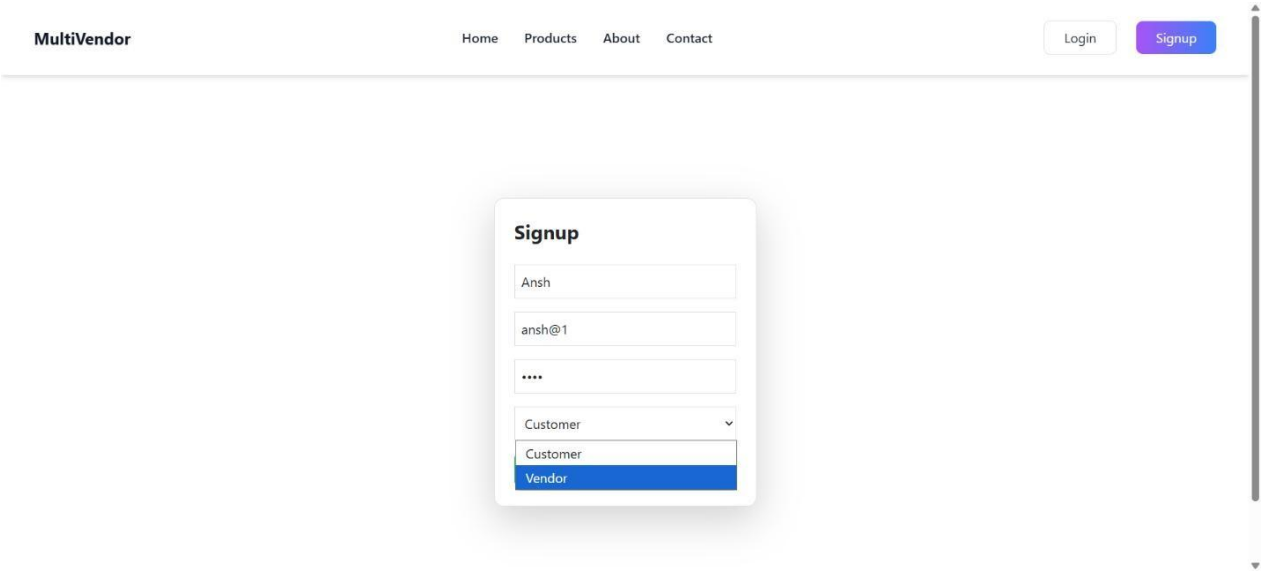


Fig H: Signing up as a vendor

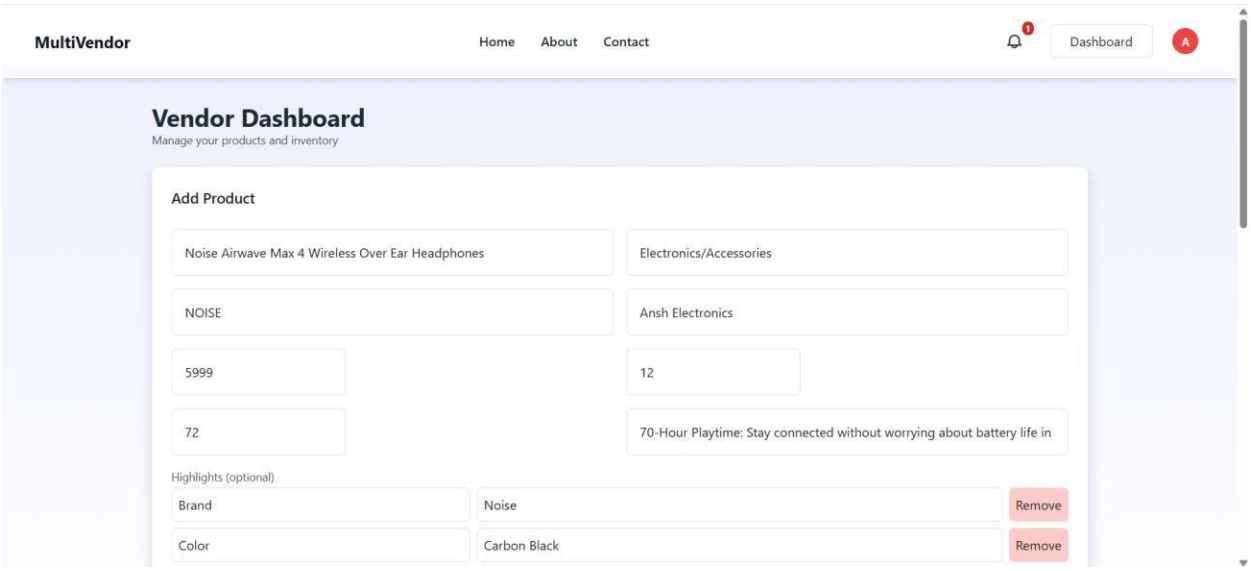


Fig I : Redirected to Vendor Dashboard, enter details.

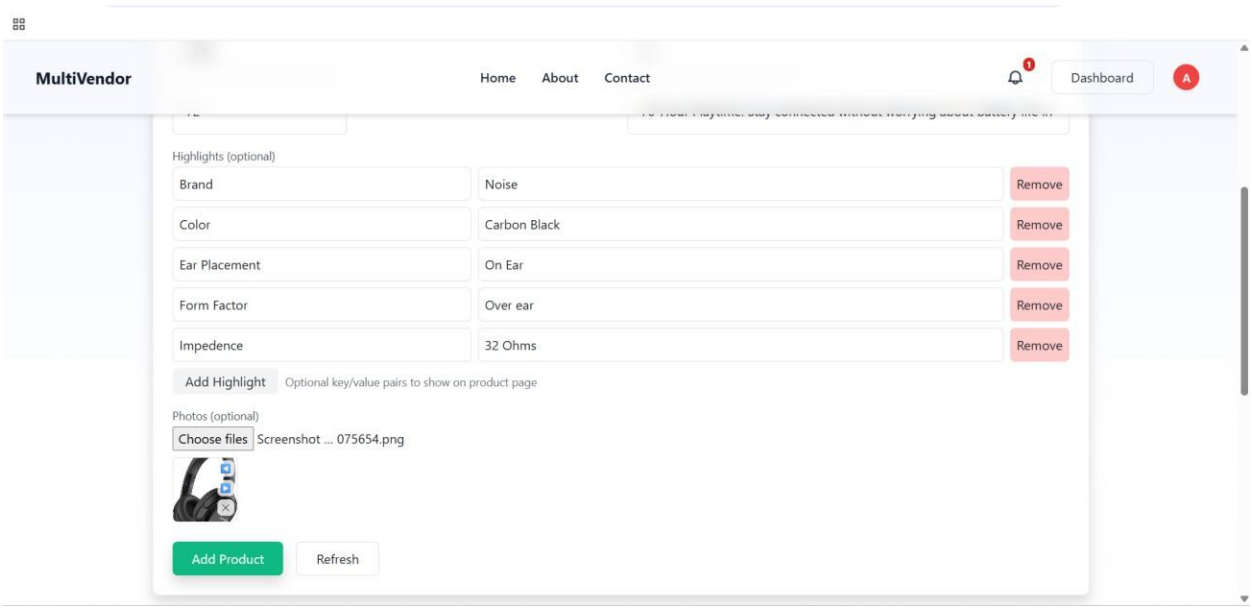


Fig J:Click Add Product

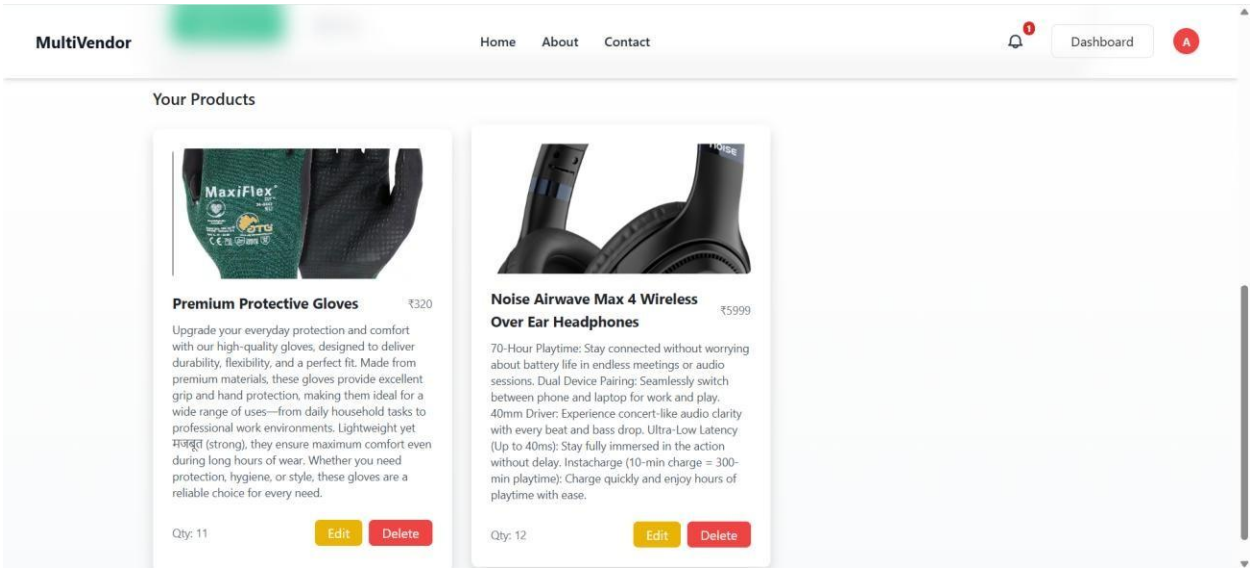


Fig K: Products visible on vendor dashboard.

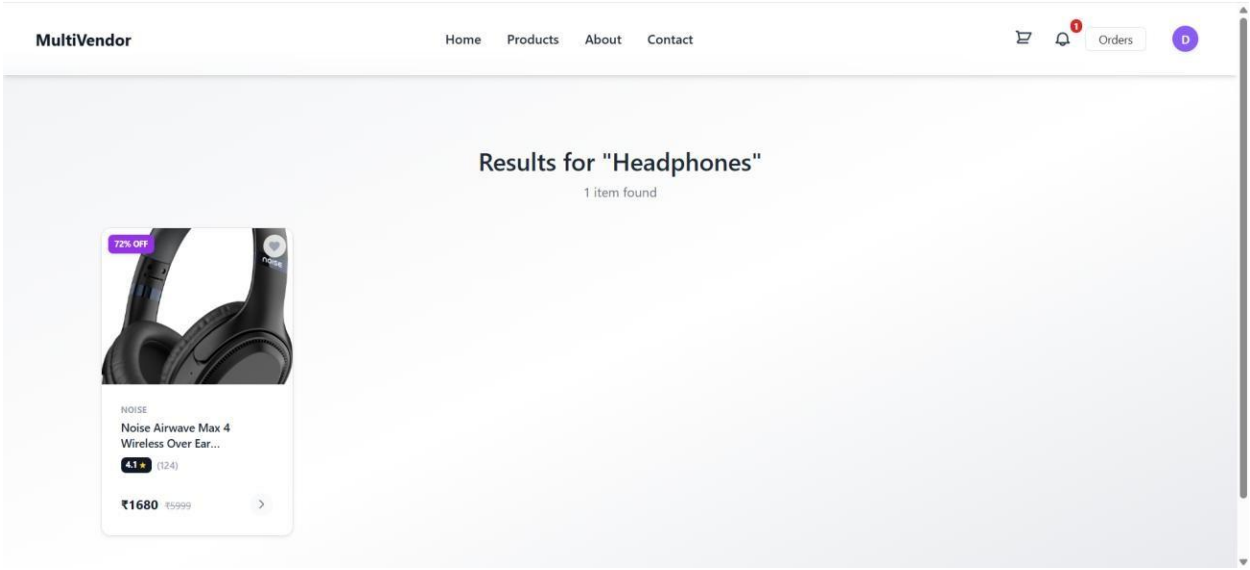


Fig L: Item visible on Customer's Dashboard

Troubleshooting Checklist

- **Product says "Out of Stock" immediately:** For vendors, please ensure the Quantity field is set greater than 0 when uploading the product.
- **Cannot Add to Cart:** Double-check that you logged in with a Customer profile, not a Vendor profile!
- **Images aren't uploading (Network Error):** Ensure your Backend terminal is running and did not crash. If the backend is off, the images cannot be saved.